

NAME: Kaif Rehman ST STD.: CSE SEC.: _____ ROLL NO.: 230701134 SUB.: Mobile Application Dev
Lab

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
		230701134		
1.	22/1/26	STRING COMPARISON	Bv	
2.	27/1/26	GUI LAYOUT WITH BUTTON AND CIRCLE BUTTON	Bv	
3.	30/1/26	APP NAVIGATION BETWEEN PAGES	Bv	
4.	5/2/26	ACTIVITY AND FRAGMENT LIFECYCLE WITH SIMPLE TIMES	Bv	
5.	12/2/26	APP ARCHITECTURE -VI LAYER WITH BUTTON AND CARD EXAMPLE	Bv	
6.	6/3/26	APP ARCHITECTURE- PERSISTENCE METHODS	Bv	
7.	12/3/26	ADVANCED RECYCLER VIEW TESTCASES	Bv	
8.	19/3/26	INTERNET CONNECTIVITY DETECTION USING CONNECTIVITY MANAGER	Bv	
9.	2/4/26	REPOSITORY PATTERN AND WORK MANAGER IMPLEMENTATION	Bv	
10	9/4/26	APP UI DESIGN PRINCIPLES	Bv	

STRING COMPARISON

Aim:

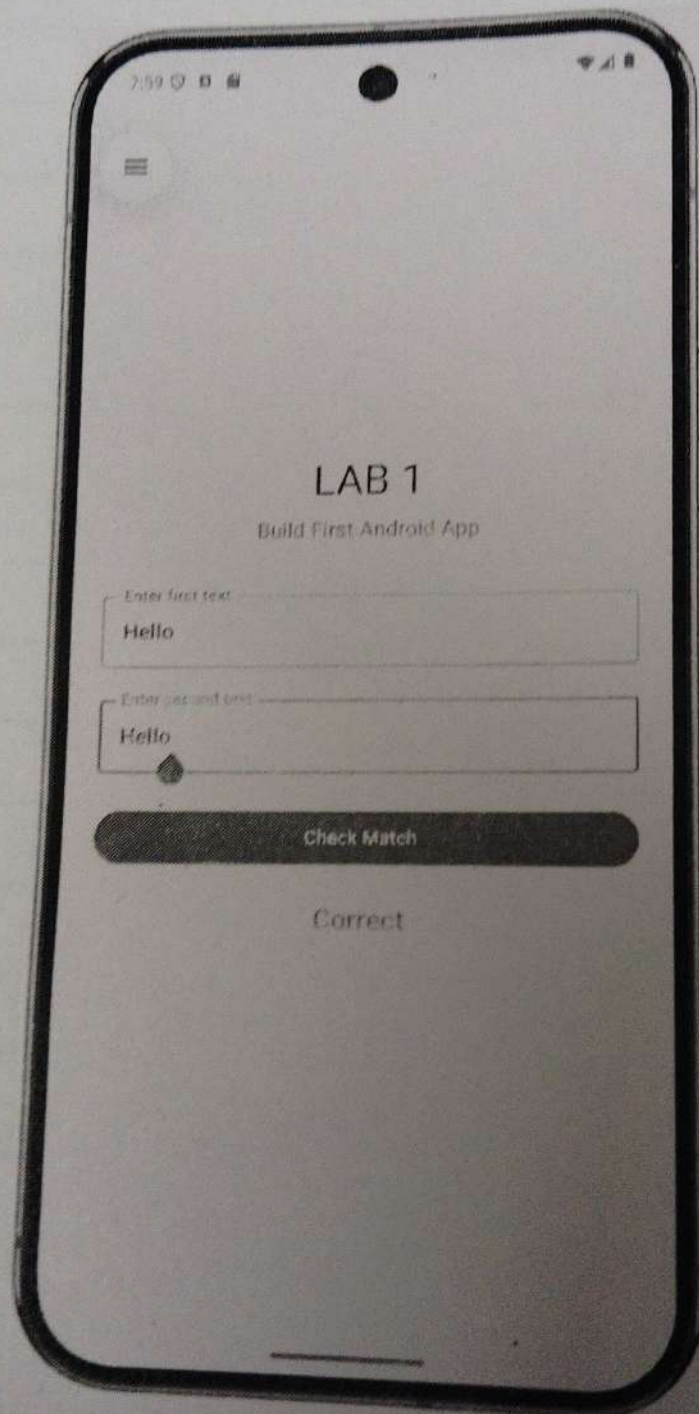
To check whether first and second text are same by considering uppercase and lowercase characters

ALGORITHM:

1. Start the Program
2. Input first text String
3. Input second text String
4. Compare String including case sensitivity
5. If both String are identical, print "Correct" else "incorrect"

RESULT:

Thus, the Program Successfully checks whether two text strings are same, considering Uppercase and Lowercase characters



GUI LAYOUT WITH LINE BUTTON AND CIRCLE BUTTON VISUALIZATION

Aim:

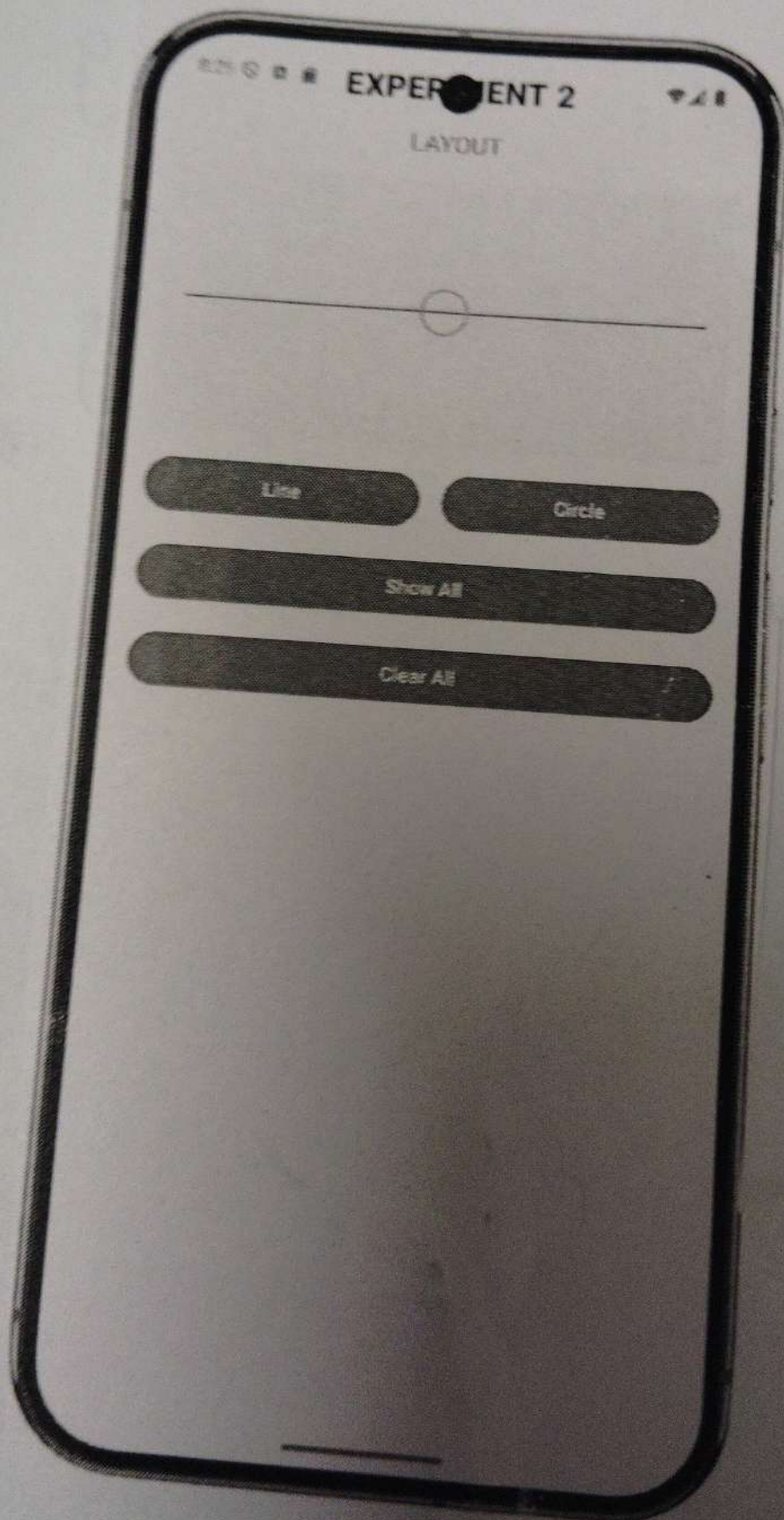
To design a graphical User interface with buttons to draw line and circle, Visualize.

ALGORITHM:

1. Start the Program and Create GUI window with layout
2. Add buttons for line, circle, Show All & Clear All
3. When line Button is pressed, generate or store line shape
4. When Circle button is pressed, generate or store circle shape
5. Use Show All - Display shapes, Clear All - remove shapes

RESULT:

Thus GUI Application successfully creates and Visualize line & circle shape by buttons.



EXPERIMENT 2

EXPERIMENT 2

LAYOUT



Line

Circle

Show All

Clear All

APP NAVIGATION BETWEEN TWO PAGES

AIM:

To implement nav between main page & second page using buttons.

ALGORITHM:

1. Start Application and create main page interface
2. Add button "Go to Second Page" on main page
3. When button is clicked, navigate to second page
4. On second page, add button "Go to Main Page".
5. When the button is clicked, navigate back to main page

RESULT:

Thus, the Application successfully performs navigation between first & second page.

8:02



LAB 3

App Navigation

Go to Second Page

Click the button above to navigate to another page

ACTIVITY AND FRAGMENT LIFECYCLE WITH SIMPLE TIMER

AIM: To demonstrate Activity & Fragment Lifecycle event with Start, Pause & Stop

ALGORITHM:

1. Start Application & Create Activity with Fragment and timer display
2. Display lifecycle events such as `onCreated()`, `onStart()` & `onResume()`
3. Add Start button to begin timer
4. Add Pause button to temporarily stop timer
5. Add Stop.

RESULT:

Thus, the program demonstrates
Activity & Fragment lifecycle with simple timer

LAB 4

Activity and Fragment Lifecycles

Simple Timer

00:00:00.00

START

PAUSE

STOP

Activity Lifecycle Events

```
[1773328228547] onCreate  
[1773328228705] onStart  
[1773328228715] onResume
```


APP ARCHITECTURE - UI LAYER WITH BUTTONS AND CARD EXAMPLE

AIM:

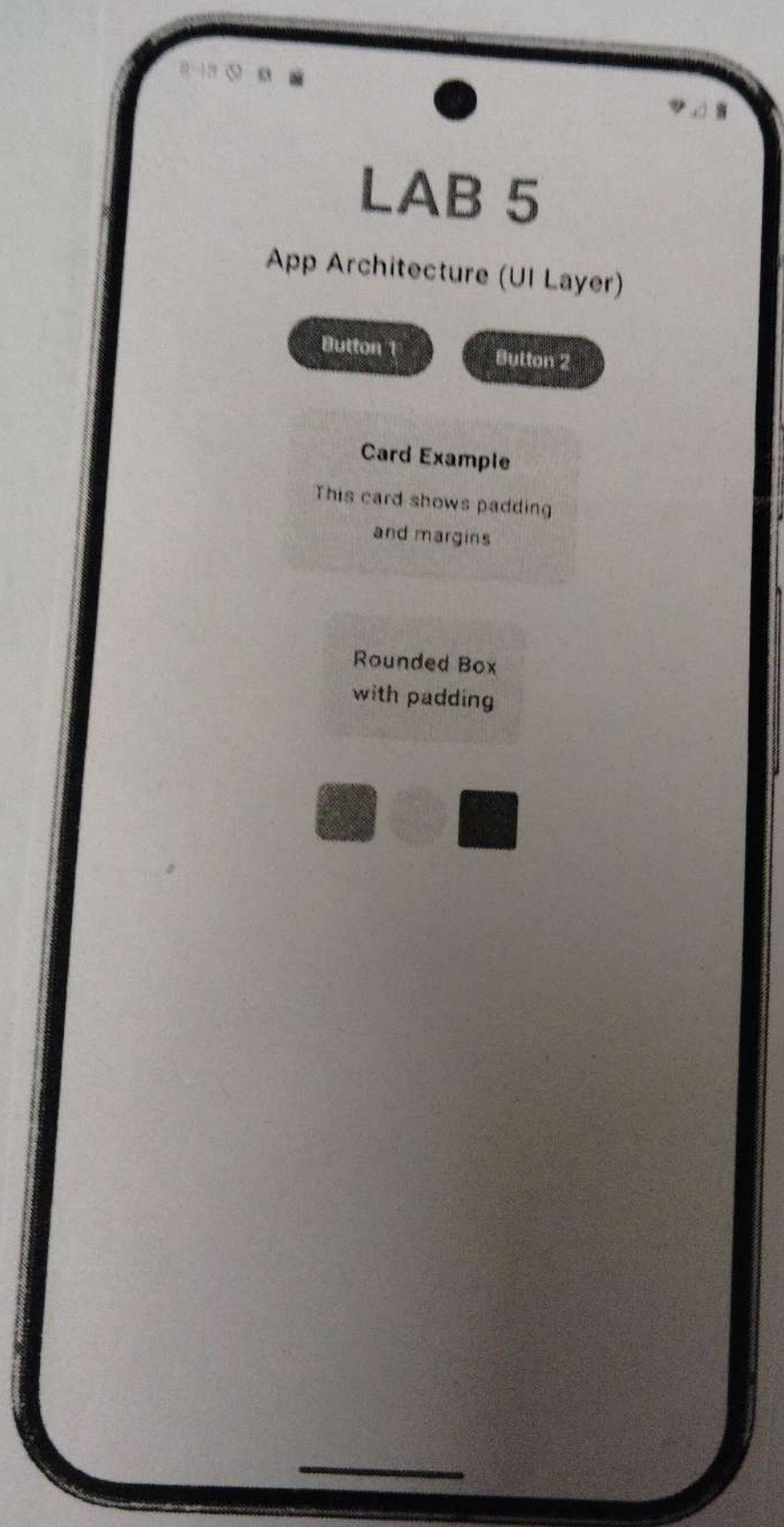
To design Simple UI layer using Buttons & Card layout with rounded box, padding & different colors.

ALGORITHM:

1. Start Application and Create UI layout
2. Add Button 1 & Button 2 to interface
3. Create Card Component
4. Apply different background colors such as red, green, blue
5. Display UI Layer with designed buttons

✓
RESULT:

Thus, Program Successfully demonstrates UI layer of app architecture using buttons



APP ARCHITECTURE - PERSISTENCE METHODS

Aim:

To demonstrate data persistence in an Application using SharedPreferences, Internal Storage & Room Database

ALGORITHM:

1. Start Application & create basic app architecture
2. Use SharedPreferences to store small Key-value data
3. Use Internal Storage to save & retrieve data files
4. Implement Room Database to store & manage tasks in Structured Database
5. Retrieve and Display Stored data using Persistence methods!

~~Result~~
RESULT:

Thus, the application Successfully demonstrates data persistence using SharedPreferences, Internal Storage & Room DB.

8:56



LAB 6

App Architecture - Persistence

SharedPreferences

Saved Username:

John Doe

Internal Storage

Saved-Note:

This is a sample note stored in internal storage.

Room Database (Tasks)

- Complete Android Lab
- Study Jetpack Compose
- Prepare Lab Screenshot

Persistence Methods Used

- SharedPreferences - Key-value storage
- Internal Storage - File based storage
- Room Database - SQLite structured storage

ADVANCED RECYCLERVIEW USE CASES

Aim:

To implement Advanced RecyclerView features including multiple view types, interactions & performance Optimizations

ALGORITHM:

1. Start Application & create basic RecyclerView using ViewBinding
2. Implement multiple view types & manage data using DiffUtil & ListAdapter
3. Add User Interaction using ItemTouchHelper & CheckListeners
4. Improve performance by optimizing data updates & view Recycling
5. Display list items

RESULT:

Thus, the Program Successfully demonstrates advanced RecyclerView features with interactions

9:02

LAB 7

Advanced RecyclerView Use Cases

GETTING STARTED



Basic RecyclerView

Setting up a simple list with a single view type



View Binding

Using ViewBinding for cleaner view access in view holders.

ADVANCED ARCHITECTURES



Multiple View Types

Combining Headers and Content items in one list.



DiffUtil & ListAdapter

Optimizing list updates with efficient background diffing

INTERACTIONS



ItemTouchHelper

Implementing Swipe-to-Delete and Drag-and-Drop functionality.



Click Listeners

Handling complex item interactions and navigation.

PERFORMANCE

INTERNET CONNECTIVITY DETECTION USING CONNECTIVITY MANAGER

Aim:

To detect Internet Connectivity & update UI when network-status changes

Algorithm:

1. Start Application
2. Use Connectivity Manager to monitor network status
3. Implement Network Callback to listen for real-time network changes
4. Detect when gain or loss internet
5. Update UI.

80
RESULT:

Thus, the Application detects internet connectivity

LAB 8

Connect to the internet



Connected

Understanding Connectivity

This app uses `ConnectivityManager` and `NetworkCallback` to listen for real-time network changes. When the device gains or loses internet access, the UI updates automatically.

REPOSITORY PATTERN AND WORK MANAGER IMPLEMENTATION

Aim:

To implement Repository Pattern & use WorkManager to perform Sync

ALGORITHM:

1. Start Application & implement Repository Pattern
2. Use WorkManager
3. Create SyncWorker to run in background
4. Execute Worker at specific time & date

~~86~~ RESULT:

Thus, Application Successfully performs background data Synchronization using WorkManager & Repository Pattern.

LAB 9

Repository Pattern and Work Manager

How it works:

1. Repository manages data from Room DB.
2. Work Manager runs SyncWorker in background.
3. UI updates automatically via Flows.

New Goal Sync

This goal was added in the background by Work Manager.

Synced at: 16:03:49, 12 Mar 2026

New Goal Sync

This goal was added in the background by Work Manager.

Synced at: 16:03:49, 12 Mar 2026

New Goal Sync

This goal was added in the background by Work Manager.

Synced at: 16:03:49, 12 Mar 2026

New Goal Sync

This goal was added in the background by Work Manager.

Synced at: 16:03:49, 12 Mar 2026

New Goal Sync

This goal was added in the background by Work Manager.

Synced at: 16:03:47, 12 Mar 2026

New Goal Sync

This goal was added in the background by

Synced at: 16:02:45, 12 Mar 2026

+ Sync Now

APP UI DESIGN PRINCIPLES

Aim:

To understand & Apply Basic UI design Principles

ALGORITHM:

1. Start Application
2. Apply Color theory to choose Consistent colors
3. Use proper Typography for readable data
4. Maintain proper layout & Spacing for better UI Structure
5. Add buttons, Cards & text-elements

RESULT:

~~130~~ Thus, Application Successfully demonstrates basic UI Design principles including Color, Typography, layout & Components

LAB 10

App UI Design

Learn & Understand UI Principles



Color Theory

Harmony, Contrast & Palette



Typography

Fonts, Scales & Readability



Layout & Spacing

Grids, Padding & Alignment



Components

Interactive UI Elements